

Predicted Questions — with Model Answers

Every question below is a prediction from the April 2023, August 2023 and March 2024 papers, filtered to your syllabus — written out with a full answer at the mark weight it carries.

LOCK repeated across papers **LIKELY** single-paper precedent or syllabus-mandated **UNIT V** no past-paper history, derived from syllabus

Exam pattern. Part A — 10 MCQs × 1 mark, no choice, exactly 2 per unit in unit order. **Part B** — 5 × 8 marks, **one full question per unit from a choice of two** (Unit I: Q1 or Q2 ... Unit V: Q9 or Q10); both sub-parts must come from the same question.

Out of syllabus — do not study: `sed`, `awk`. They appear in the past papers but not in your syllabus.

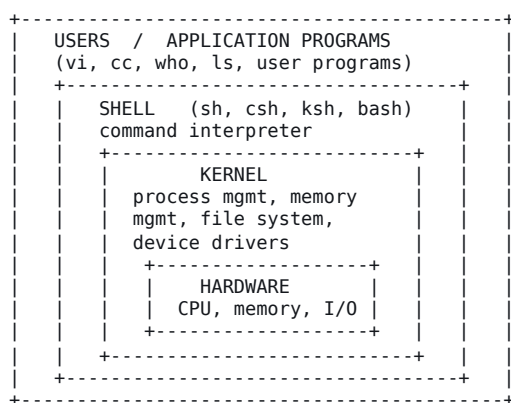
UNIT I — Introduction to UNIX

PART B · Q1 or Q2 · 8 marks

1. Explain with a figure the kernel and shell relationship in UNIX / Illustrate the layered UNIX architecture with a neat diagram.

LOCK Part B · Q1(a) · 4–5 marks · Apr 2023 + Mar 2024

UNIX has a **layered architecture**. Each layer talks only to the layer directly beneath it, so the user is insulated from the hardware.



Kernel

- The **core of the operating system**. Loaded into memory at boot time and stays resident until shutdown.
- Manages **memory, processes, the file system and devices**. Schedules the CPU among processes.
- **Only the kernel communicates directly with the hardware**. No user program touches hardware itself.
- Programs request kernel services through **system calls** (`open`, `read`, `write`, `fork`, `exec`). This is the kernel's only interface to the outside.
- There is exactly **one** kernel on a machine.

Shell

- The **command interpreter** — the interface between the user and the kernel. It is **not** part of the kernel; it is an ordinary user program.
- Reads the command typed at the prompt, interprets it, and has the kernel execute it, then displays the result.
- Because it is a user program, **many shells can exist** and each user can choose one: **Bourne shell (sh)**, **C shell (csh)**, **Korn shell (ksh)**, **Bash**.
- Also a **programming language** — commands can be collected into a shell script.

Shell interpretive cycle

The shell repeats this loop for every command: **display prompt** → **read the command line** → **parse it (split into words, expand wildcards and variables)** → **locate the command in PATH** → **fork a child and exec the command** → **wait for it to finish** → **display the prompt again**.

Marks tip: the diagram alone is worth about half. Draw it, label all four layers, then write two or three lines each on kernel and shell. Mention *system calls* and *shell interpretive cycle* by name.

2. What are file permissions? Explain the use of `chmod` to change file permissions using both the absolute and the relative methods.

LOCK Part B · Q2 · 3–4 marks · Part B 2 of 3 papers, Part A 3 of 3

Every UNIX file has **nine permission bits**, arranged as three sets of three. The sets are **owner (user)**, **group**, **others**; within each set the bits are **r (read)**, **w (write)**, **x (execute)**.

```
$ ls -l note.txt
-rwxr-xr-- 1 jnan cse 120 Sep 17 10:20 note.txt
|_|_|_|_|
| | +--- others : r-- (read only)
| +----- group : r-x (read, execute)
+----- owner : rwx (read, write, execute)
```

Permission	File	Directory
r (read)	display its contents	list the names in it (<code>ls</code>)
w (write)	modify its contents	create or delete files inside it
x (execute)	run it as a program	<code>cd</code> into it

Method 1 — Absolute (octal) method

Each permission has a weight: **r = 4**, **w = 2**, **x = 1**. Add the weights for each of the three sets to get a three-digit octal number.

Octal	Permissions	Meaning
<code>chmod 755 f</code>	<code>rwX r-X r-X</code>	owner all; group and others read + execute
<code>chmod 644 f</code>	<code>rw- r-- r--</code>	owner read+write; everyone else read only
<code>chmod 744 f</code>	<code>rwX r-- r--</code>	owner all; group and others read only
<code>chmod 777 f</code>	<code>rwX rwX rwX</code>	everyone everything
<code>chmod 600 f</code>	<code>rw- --- ---</code>	private to the owner

Absolute is "set exactly this" — it replaces all nine bits, whatever they were before.

Method 2 — Relative (symbolic) method

The expression has **three components: who + operator + permission**.

Component	Values
Who	<code>u</code> user/owner · <code>g</code> group · <code>o</code> others · <code>a</code> all three
Operator	<code>+</code> add · <code>-</code> remove · <code>=</code> set exactly
Permission	<code>r</code> · <code>w</code> · <code>x</code>
<code>chmod u+x script.sh</code> add execute for the owner <code>chmod go-w note.txt</code> remove write from group and others <code>chmod a=r note.txt</code> everyone gets read only <code>chmod u+x,go-r file</code> two expressions, separated by a comma	

Relative is "change only what I name" — bits not mentioned keep their old value.

Also worth a line: default permission for an ordinary file is **666 (rw-rw-rw-)** and for a directory **777**; the actual permission is these minus the `umask` value (typically 022, giving 644 and 755). Only the **owner** or **root** may change a file's permissions.

3. Explain the following commands with syntax, options and an example. *(The list changes every year — this covers the whole inventory.)*

LOCK Part B · Q1(b) or Q2(a) · 4–5 marks · 3 of 3 papers

Answer each command in the same three-part shape: **what it does** → **syntax with two options** → **one worked example**.

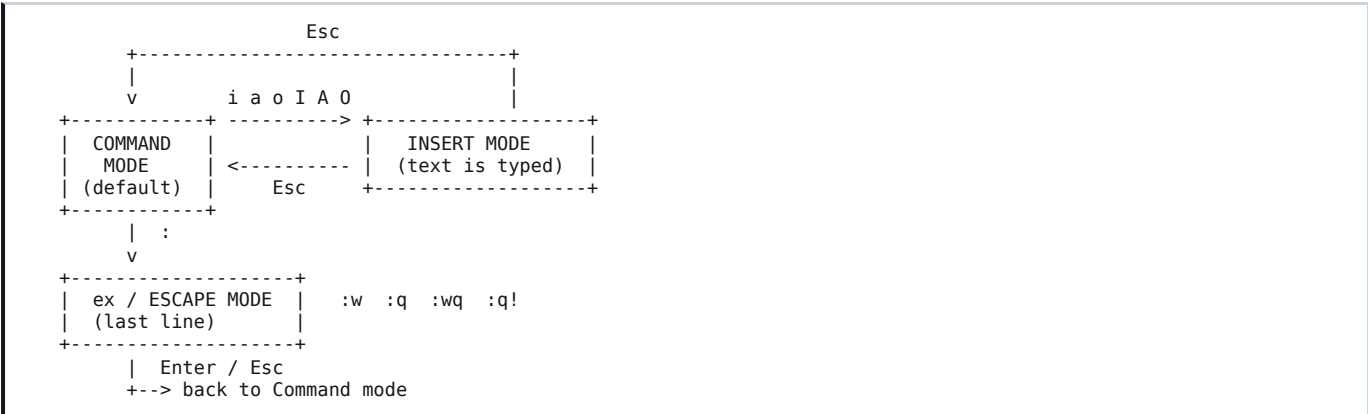
Command	Purpose & syntax	Key options	Example
echo	Display text / variable value. echo [options] string	-n no trailing newline -e enable escapes (\n , \t)	echo -n "Name: "
printf	Formatted output, like C. printf "format" args	%s string, %d integer	printf "%s is %d\n" Sam 20
ls	List directory contents. ls [options] [dir]	-l long listing · -a include hidden -R recursive · -t by time	ls -la /home
who	Users currently logged in. who [options]	-H print header who am i your own line	who -H
date	Show/set system date and time. date [+format]	+%d-%m-%Y · +%H:%M:%S	date "+%d-%m-%Y"
passwd	Change your login password. passwd [user]	prompts for old then new password	passwd
uname	System information. uname [options]	-a all · -r kernel release	uname -a
pwd	Print working directory.	(no options needed)	pwd
cd	Change directory. cd [dir]	cd .. parent · cd alone → home	cd /usr/bin
mkdir	Create directory. mkdir [-p] dir	-p create parent directories too	mkdir -p a/b/c
rmdir	Remove an empty directory.	fails if the directory is not empty	rmdir olddir
cp	Copy files. cp [options] src dest	-i prompt before overwrite -r recursive · -p preserve attributes	cp -i f1 f2
mv	Move or rename a file. mv [options] src dest	-i interactive · -f force	mv old.txt new.txt
rm	Delete files. rm [options] file	-i confirm · -r recursive · -f force	rm -i note.txt
cat	Display / create / concatenate files. cat [options] file(s)	-n number lines · -s squeeze blanks	cat f1 f2 > f3
more	Display a file one screen at a time.	Space = next page, q = quit	more bigfile.txt
wc	Count lines, words, characters. wc [options] file	-l lines · -w words · -c characters	wc -l file.txt
man	Display the manual page for a command.	man -k keyword search	man ls
script	Record a terminal session into a file.	ends with <code>exit</code> ; default file typescript	script session.log
stty	Set/display terminal settings.	-a show all settings	stty -a
od	Octal dump — view a file's bytes.	-b octal · -c characters · -x hex	od -c file.txt
lp	Send a file to the printer.	-d printer name · -n copies	lp -n 2 report.txt
tar	Archive many files into one. tar [options] archive files	-c create · -x extract · -t list -v verbose · -f archive filename	tar -cvf bk.tar dir/

Command	Purpose & syntax	Key options	Example
gzip	Compress a file (.gz).	gunzip decompress · zcat view	gzip bk.tar
PATH	Environment variable holding the list of directories the shell searches for commands.	colon-separated list	echo \$PATH

4. Illustrate the working of the vi editor with its different modes.

LIKELY Part B · Q1(a) · 4 marks · Aug 2023 · explicitly named in your syllabus

vi (visual editor) is the standard full-screen text editor of UNIX. It works in **three modes**, and the same keystroke means different things in each — this is why mode awareness matters.



Mode	Entered by	What it is for
1. Command mode	Default on opening vi file; press Esc from any mode	Navigation and editing. Keys are commands, not text. h j k l left/down/up/right · dd delete line · yy copy (yank) line · p paste · x delete a character · u undo · G last line
2. Insert mode	i insert before cursor · a append after · o open line below · I A O variants	Whatever you type becomes text in the file. Leave with Esc .
3. ex / Escape mode	Type : in command mode — the cursor drops to the last line	File-level commands. :w save · :q quit · :wq save and quit · :q! quit discarding changes · :set nu line numbers · :/pattern search

Marks tip: draw the three-box mode diagram with the transition keys on the arrows. Naming **Esc** as the universal way back to Command mode is the point the examiner looks for.

5. List and explain the salient features of the UNIX operating system.

LIKELY Part B · Q1(b) · 4 marks · Mar 2024 (most recent paper)

Feature	Explanation
Multi-user	Many users can log in and work on the same machine at the same time; the kernel keeps each user's files and processes separate.
Multitasking	One user can run several programs at once — one in the foreground, others in the background with & . The kernel time-shares the CPU.
Portability	Written almost entirely in C , not assembly, so UNIX can be moved to a new machine by recompiling. This is the main reason it spread.
Hierarchical file system	A single inverted tree rooted at / . Everything — including devices — is treated as a file.
Security	Login name and password, plus the three-level (owner/group/others) file permission system, plus file ownership.

Feature	Explanation
The shell	A programmable command interpreter. Users can write shell scripts, so the interface itself is extensible.
Pipes and filters	The building-block approach : each tool does one small job well, and pipes join them into larger jobs — <code>who wc -l</code> .
Communication	Utilities for messaging and networking between users and machines — <code>mail</code> , <code>write</code> , <code>ssh</code> , <code>ftp</code> .
Large tool set	Hundreds of small general-purpose utilities supplied with the system.

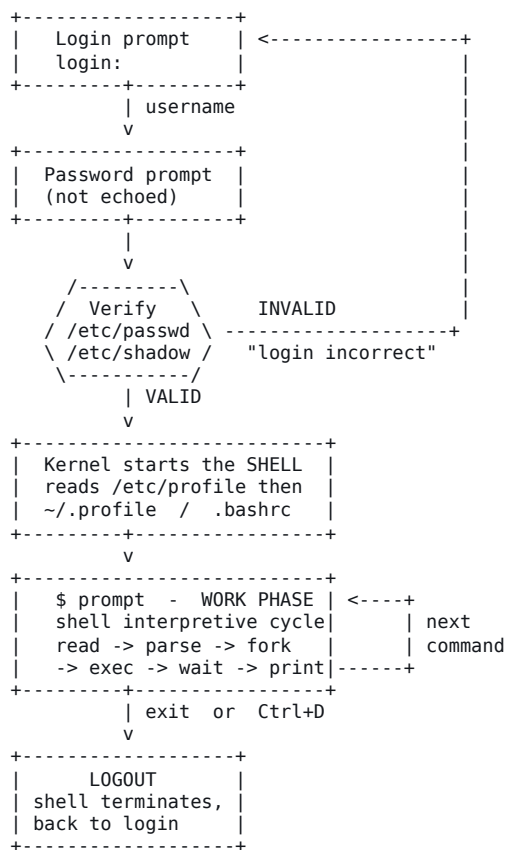
UNIT II — Shells & Utilities

PART B · Q3 or Q4 · 8 marks | ★ ANSWER Q3

6. Illustrate the various stages of a UNIX session with a neat diagram.

HIGHEST CONFIDENCE Part B · Q3(a) · 4–5 marks · ALL 3 PAPERS, same slot, near-identical wording

A UNIX session is everything between logging in and logging out. It has **three stages**: **Login** → **Work (the session proper)** → **Logout**.



1 · Login stage

- The `getty` process displays the **login:** prompt on the terminal.
- The user enters a **username**, then a **password** — which is **not echoed** to the screen.
- `login` verifies the pair against `/etc/passwd` (user details) and `/etc/shadow` (encrypted passwords). The entered password is encrypted and the result compared — the stored password is never decrypted.
- If verification **fails**, the message *login incorrect* appears and control returns to the login prompt.

2 · Work phase (the session)

- On success the kernel starts the user's **login shell**, named in the last field of that user's `/etc/passwd` entry.
- The shell runs the system profile `/etc/profile`, then the user's own `~/.profile` (or `.bash_profile` / `.bashrc`) — this is where `PATH`, prompts and aliases get set.
- The user is placed in their **home directory** and the **\$ prompt** appears.
- The shell now repeats the **shell interpretive cycle** for every command: display prompt → read the line → parse and expand → locate the command → `fork` and `exec` → wait → display output → prompt again.

3 · Logout stage

- The user types `exit` or presses **Ctrl+D** (end-of-file) at the prompt.
- The login shell terminates, the session's processes are cleaned up, and `getty` redisplay the login prompt for the next user.

Marks tip: this is the single most repeated question on the paper. Draw the flow with the **invalid-password loop back to the login prompt** — that arrow is what separates a full-mark diagram from a half-mark one. Name `/etc/passwd`, `.profile` and **Ctrl+D** explicitly.

7. Discuss process utilities — the types of process, process initialization and termination, and how `nice` and `renice` adjust the priority of a running process.

LOCK Part B · Q3(b) · 4–5 marks · 3 of 3 papers, angle rotates — all three covered here

A **process** is a program in execution. Every process has a unique **PID**, a parent (**PPID**), an owner, and a priority.

Types of process

Type	Description
Interactive / foreground	Started from the terminal and holds it — the shell waits for it to finish before returning the prompt.
Background	Started with <code>&</code> . Runs while the shell returns the prompt immediately: <code>sort big.txt &</code>
Daemon	A system process that runs continuously in the background, not attached to any terminal — <code>init</code> , <code>cron</code> , <code>lpd</code> , <code>inetd</code> .
Zombie	Has finished executing, but its parent has not yet collected its exit status, so its entry remains in the process table.
Orphan	Its parent died first. It is adopted by <code>init</code> (PID 1), which becomes its new parent.

Process initialization (creation)

- `fork()` — a process creates a near-exact copy of itself. The copy is the **child**; it gets a new PID, and `fork` returns 0 to the child and the child's PID to the parent.
- `exec()` — the child then overlays its own memory image with the new program to be run. **No new process is created** by `exec`; the same PID now runs different code.
- `wait()` — the parent suspends itself until the child finishes and picks up its exit status.
- This is the **fork-and-exec** mechanism: it is how the shell runs every external command.
- **`init` is PID 1**, started by the kernel at boot, and is the ancestor of every other process. Boot sequence: `init` → `getty` → `login` → `shell`.

Process termination

A process ends by calling `exit(n)`, returning an **exit status** `n` to its parent (**0 = success**). It can also be ended from outside with `kill PID` (SIGTERM, polite) or `kill -9 PID` (SIGKILL, cannot be refused).

Priority — `nice` and `renice`

- Every process carries a **nice value from -20 to +19**, default **0**. A process is "nice" to others when it gives up CPU — so **-20 = highest priority, +19 = lowest**.
- `nice` starts a **new** command at a changed priority:
`nice -n 10 sort big.txt` — runs with priority lowered by 10.
- `renice` changes the priority of an **already running** process, identified by PID:
`renice -n 5 -p 4312` — lowers the priority of process 4312.
- **Only the superuser (root) can raise priority** (use a negative nice value). An ordinary user may only lower their own processes' priority.
- View priorities with `ps -l` (NI column) or `top`.

Related process commands

`ps` show process attributes (`-f` full, `-e` every process, `-u` user) · `kill` terminate · `jobs` list jobs of this shell · `&` run in background · `wait` · `sleep n`.

8. Explain the `du` command with its different options. / Name and explain UNIX commands used for text processing. / Briefly explain the networking commands.

LIKELY Part B · Q4 · 3–5 marks · du 2 of 3, text processing 2 of 3, networking Mar 2024

Disk utilities — `du` and `df`

`du` (**disk usage**) reports how much disk space files and directories occupy, walking the directory tree.

Option	Meaning & example
<code>-a</code>	Report every file , not just directories — <code>du -a /home/jnan</code>
<code>-s</code>	Summary — one grand total per argument — <code>du -s /home/jnan</code>
<code>-h</code>	Human readable — K, M, G instead of blocks — <code>du -h dir</code>
<code>-k</code>	Report sizes in kilobytes
<code>-c</code>	Print a grand total at the end

`df` (**disk free**) reports free and used space per mounted file system: `df -h` human readable, `df -i` inode usage. **Difference:** `du` measures what files consume, `df` measures what the file system has left.

Text processing utilities

Command	Purpose	Example
<code>head</code>	First n lines of a file	<code>head -5 emp.lst</code>
<code>tail</code>	Last n lines of a file	<code>tail -5 emp.lst</code>
<code>cut</code>	Extract columns or fields	<code>cut -d: -f1 /etc/passwd</code>
<code>paste</code>	Join files side by side	<code>paste f1 f2</code>
<code>sort</code>	Sort lines	<code>sort -n marks.txt</code>
<code>uniq</code>	Handle duplicate adjacent lines	<code>sort f uniq -d</code>
<code>tr</code>	Translate or delete characters	<code>tr 'a-z' 'A-Z' < f</code>
<code>wc</code>	Count lines, words, characters	<code>wc -l f</code>
<code>grep</code>	Search lines matching a pattern	<code>grep "unix" f</code>

Networking commands

Command	Purpose
<code>ping</code>	Tests whether a host is reachable , by sending ICMP echo-request packets and timing the replies. <code>ping google.com</code>
<code>host</code>	DNS lookup — converts a hostname to an IP address and back. <code>host google.com</code>
<code>tracpath</code> / <code>tracroute</code>	Shows the route packets take to a host, listing every intermediate hop and its delay.
<code>netstat</code>	Displays network connections , listening ports, routing tables and interface statistics. <code>netstat -r</code>
<code>route</code>	Manipulates the TCP/IP routing table — shows, adds or deletes routes.
<code>ifconfig</code>	Configures or displays network interface parameters (IP address, netmask).
<code>ftp</code>	File Transfer Protocol — transfers files between hosts (<code>get</code> , <code>put</code> , <code>mget</code>).
<code>telnet</code> / <code>ssh</code>	Log in to a remote machine. ssh is encrypted; telnet is not — prefer <code>ssh</code> .
<code>finger</code>	Displays information about a user (login name, home directory, last login).

Backup utilities — `tar` and `gzip`

`tar` bundles many files into one archive (it does not compress): `-c` create, `-x` extract, `-t` list contents, `-v` verbose, `-f` name the archive file.

`tar -cvf backup.tar project/` → create · `tar -xvf backup.tar` → extract · `tar -tvf backup.tar` → list

`gzip` compresses a single file, replacing it with `.gz`: `gzip backup.tar` → `backup.tar.gz`; `gunzip` restores it; `zcat` views it without unpacking. Combined: `tar -zcvf backup.tar.gz project/` .

9. Explain the three standard streams / standard redirection files with respect to UNIX. What is redirection?**HIGHEST CONFIDENCE** Part B · Q5(a) · 3–5 marks · ALL 3 PAPERS, same slot

When any UNIX command runs, the kernel opens **three files automatically** for it. These are the **standard streams**, and each has a fixed **file descriptor** number.



Stream	fd	Default source/target	Purpose
Standard input (stdin)	0	Keyboard	Where the command reads its input from
Standard output (stdout)	1	Terminal / screen	Where normal results are written
Standard error (stderr)	2	Terminal / screen	Where error messages are written — kept separate so errors don't pollute redirected output

Redirection

Redirection is the shell's ability to detach a standard stream from its default file and attach it to a file of the user's choosing. The command itself is unchanged and unaware — it still just reads fd 0 and writes fd 1. The shell does the reconnection before the command starts.

Operator	Meaning	Example
<code>></code>	Output redirection — creates or overwrites	<code>ls -l > list.txt</code>
<code>>></code>	Output redirection — appends to the file	<code>date >> log.txt</code>
<code><</code>	Input redirection — read from a file instead of the keyboard	<code>wc -l < list.txt</code>
<code>2></code>	Redirect errors only	<code>ls xyz 2> err.log</code>
<code>2>></code>	Append errors	<code>cmd 2>> err.log</code>
<code>> f 2>&1</code>	Send both output and errors to the same file	<code>cmd > all.log 2>&1</code>
<code>/dev/null</code>	The "bit bucket" — anything written to it is discarded	<code>cmd 2> /dev/null</code>

```

$ ls xyz          # xyz does not exist
ls: cannot access 'xyz': No such file or directory    <- goes to stderr (2)

$ ls xyz > out.txt # only stdout redirected; the error STILL appears on screen
ls: cannot access 'xyz': No such file or directory

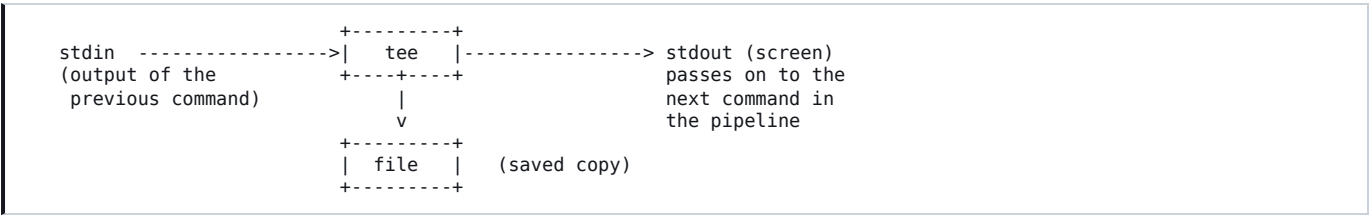
$ ls xyz 2> err.txt # now the error goes to the file, screen stays clean
  
```

Marks tip: the example above — showing that `>` alone does **not** capture errors — is the cleanest way to demonstrate why stdout and stderr are separate streams. Examiners give credit for it.

10. Demonstrate the working of the tee command with a necessary diagram. Explain how tee can be integrated into a pipeline.

LOCK Part B · Q5(b) or Q6(a) · 4–5 marks · 2 of 3 papers + a Part A MCQ

tee reads from standard input and writes simultaneously to standard output and to one or more files. It is named after the plumbing T-junction: the data stream arrives and leaves in two directions at once.



Syntax: command | tee [-a] filename | next-command — -a appends instead of overwriting.

Why it is needed

A pipe sends output onward only — once data flows into the next command it is gone, so you cannot see or save the intermediate result. Redirection with > does the opposite: it captures the output but ends the pipeline. tee solves both: it saves a copy and lets the data continue.

```
$ ls -l | tee list.txt | wc -l
12
           # list.txt now holds the full listing,
           # and wc -l still received it and printed the count

$ who | tee users.txt | grep "jnan"
           # saves everyone logged in, but displays only jnan's line

$ ls -l | tee -a log.txt | wc -l
           # appends to log.txt instead of overwriting it

$ cat file | tee copy1 copy2 copy3
           # tee can write to several files at once
```

Marks tip: draw the T-junction, give the three-stage pipeline example, and state the one-line reason: a pipe passes data on but does not save it; > saves data but stops the pipeline; tee does both.

11. Explain the types of variables in UNIX with examples. What are environment variables?

LIKELY Part B · Q5(b) or Q6(b) · 3–4 marks · Apr 2023 · two separate syllabus lines

A shell variable is a name holding a value. Assignment rule: no spaces around the = sign. Read a variable's value by prefixing \$.

```
$ x=10      # correct
$ x = 10    # WRONG - shell reads x as a command
$ echo $x
10
```

Type	Description
1. Local (shell) variables	Created by the user in the current shell. Exist only in that shell — a child process or sub-shell cannot see them. name=Sam ; echo \$name
2. Environment variables	Local variables that have been exported. They are passed to every child process the shell creates, so a script can read them. By convention written in UPPERCASE.
3. Shell / system variables	Set by the system at login, controlling how the shell behaves — PATH , HOME , PS1 .
4. Positional parameters	Set automatically from command-line arguments — \$0 \$1 \$2 ... \$# \$* \$? (see Unit V).

Important environment variables

Variable	Holds
PATH	Colon-separated list of directories the shell searches for commands
HOME	Your home directory — where cd with no argument takes you

Variable	Holds
SHELL	Full path of your login shell
USER / LOGNAME	Your login name
PS1	Primary prompt string (normally <code>\$</code>)
PS2	Secondary/continuation prompt (normally <code>></code>)
TERM	Terminal type — used by <code>vi</code> and other full-screen programs
MAIL	Path of your mailbox file
<pre> \$ echo \$HOME /home/jnan \$ echo \$PATH /usr/local/bin:/usr/bin:/bin \$ set list ALL variables \$ env list only environment variables \$ x=10 create a local variable \$ export x promote it to an environment variable \$ export y=20 create and export in one step \$ unset x remove a variable \$ readonly z=5 make it unchangeable </pre>	

The key distinction for marks: a **local** variable dies with the current shell and is invisible to child processes; an **environment** variable is inherited by every child process. `export` is what converts one into the other.

12. What is an alias? Explain creating and removing aliases. Explain the need for command substitution with an example.

LIKELY Part B · Q6(b) · 3–4 marks · Aug 2023 (alias) + Apr 2023 (command substitution)

Aliases

An **alias** is a **short, user-defined name for a long or frequently used command**, expanded by the shell before execution. It saves typing and lets you attach default options permanently.

```

$ alias ll='ls -l'           create an alias
$ alias rm='rm -i'           make deletion always ask first (a safety alias)
$ alias c=clear
$ alias                      list all defined aliases
$ alias ll                   show the definition of one alias
$ unalias ll                 remove an alias
$ \ls                       bypass an alias for one run (backslash)

```

- **No spaces around the `=`**, and quote the command if it contains spaces or options.
- Aliases defined at the prompt vanish when the shell exits — to make them permanent, put them in `~/.profile` or `~/.bashrc`.

Command substitution

Command substitution replaces a command with its own output, so the result of one command can be used inside another command or assigned to a variable.

Two forms: backquotes ``command`` (older) or `$(command)` (modern, and nestable).

```

$ echo "Today is date"      # WRONG - prints the word "date" literally
Today is date

$ echo "Today is `date`"    # command substitution
Today is Wed Sep 17 10:30:00 IST 2025

$ echo "You have $(ls | wc -l) files in this directory"
You have 12 files in this directory

$ today=$(date +%d-%m-%Y)   # store output in a variable
$ echo $today
17-09-2025

$ users=`who | wc -l`
$ echo "$users users are logged in"

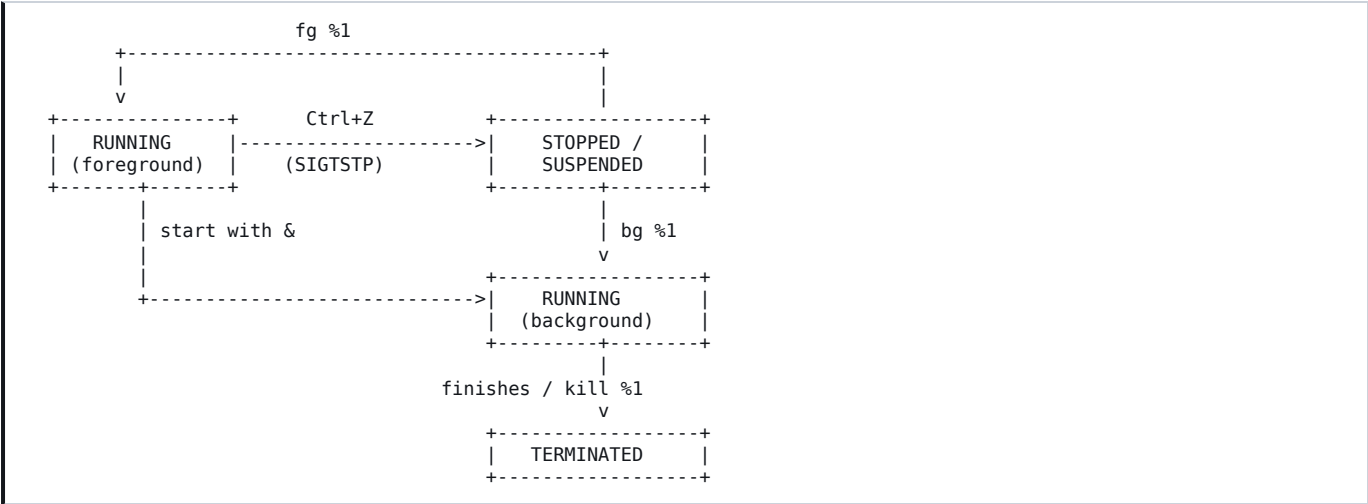
```

Why it is needed: without it, a command's output can only go to the screen or a file. Command substitution lets that output become **part of another command line** — the shell runs the inner command first, drops its output in place, and only then runs the outer command. It is essential in shell scripts for storing computed results.

13. Illustrate the different job states in UNIX with a neat diagram. Demonstrate how `bg` and `fg` move a job between background and foreground. Examine the role of a signal such as `SIGSTOP` in job control.

LIKELY Appeared in all 3 papers · but your syllabus files this under Unit II process utilities — prepare it, expect it one unit earlier

Job control is the shell's ability to run several jobs at once and move them between foreground, background and suspended states. A **job** is a command (or pipeline) started from the shell, given a **job number** referred to as `%1`, `%2`, ...



Command	Action
command &	Start the job directly in the background ; the prompt returns at once
Ctrl+Z	Suspend the foreground job (sends SIGTSTP) — it stops but is not killed
bg %1	Resume a stopped job, running it in the background
fg %1	Bring a background or stopped job into the foreground
jobs	List this shell's jobs with their job numbers and states
kill %1	Terminate job 1

Signals used in job control

Signal	Key	Effect
SIGINT (2)	Ctrl+C	Interrupt — asks the process to terminate. Can be caught or ignored.
SIGTSTP (20)	Ctrl+Z	Terminal stop — suspends the job. Can be caught , so a program may refuse it.
SIGSTOP (19)	—	Stop the process. Cannot be caught, blocked or ignored — this is what guarantees job control always works, even against a program that traps Ctrl+Z.
SIGCONT (18)	—	Continue a stopped process — this is what <code>bg</code> and <code>fg</code> send.
SIGKILL (9)	—	<code>kill -9</code> — immediate termination, cannot be caught. Last resort.

```
$ sort bigfile.txt          # running in foreground
^Z                          # Ctrl+Z - suspend it
[1]+  Stopped    sort bigfile.txt
$ bg %1                   # resume it in the background
[1]+  sort bigfile.txt &
$ jobs
[1]+  Running    sort bigfile.txt &
$ fg %1                   # pull it back to the foreground
```

The SIGSTOP point that earns the mark: SIGTSTP (Ctrl+Z) can be caught and ignored by a program, but **SIGSTOP cannot be** — so the system always retains the ability to suspend any process.

14. Describe the five categories of operators commonly used in regular expressions for matching text patterns.**LOCK · FIXED FORMULA** Part B · Q7 or Q8 · 5 marks · appeared twice, 5 marks both times, same phrasing

A **regular expression** is a pattern describing a set of strings, using ordinary characters plus **metacharacters** that have special meaning. Its operators fall into five categories.

1 · Atoms — *what* to match

The smallest unit a pattern can match.

Atom	Matches	Example
literal char	Itself	grep "unix" f
<code>.</code> (dot)	Any single character	b.t → bat, bit, but
<code>[abc]</code>	Any one character from the set	[bch]at → bat, cat, hat
<code>[^abc]</code>	Any one character not in the set	[^0-9] → any non-digit
<code>[a-z]</code>	Any one character in the range	[A-Z] → any capital
<code>\</code>	Escapes a metacharacter, making it literal	\. → a real full stop

2 · Quantifiers (repetition operators) — *how many*

Operator	Meaning	Example
<code>*</code>	Zero or more of the immediately preceding character	ab*c → ac, abc, abbc
<code>+</code>	One or more of the preceding (<i>ERE</i> — <i>needs egrep</i>)	ab+c → abc, abbc (not ac)
<code>?</code>	Zero or one of the preceding (<i>ERE</i>)	colou?r → color, colour
<code>{m,n}</code>	Between m and n occurrences	[0-9]{3} → exactly 3 digits

3 · Anchors (position operators) — *where* in the line

Operator	Meaning	Example
<code>^</code>	Match at the beginning of the line	grep "^A" f → lines starting with A
<code>\$</code>	Match at the end of the line	grep "ed\$" f → lines ending in "ed"
<code>^...\$</code>	Match the whole line	grep "^\$" f → empty lines

4 · Alternation and grouping

Operator	Meaning	Example
<code> </code>	OR — match either alternative (<i>ERE</i>)	egrep "cat dog" f
<code>()</code>	Group a sub-expression so an operator applies to all of it	egrep "(ab)+" f → ab, abab

5 · Character classes (POSIX classes)

Readable names for common sets, written inside a further pair of brackets.

Class	Matches
<code>[:alpha:]</code>	Any letter
<code>[:digit:]</code>	Any digit (same as <code>[0-9]</code>)
<code>[:alnum:]</code>	Any letter or digit
<code>[:upper:]</code> / <code>[:lower:]</code>	Any uppercase / lowercase letter
<code>[:space:]</code>	Space, tab or newline
<code>[:punct:]</code>	Any punctuation character

```
$ grep "^[A-Z].*[0-9]$" emp.lst
# lines starting with a capital AND ending with a digit

$ grep "$" file | wc -l          count the empty lines
$ egrep "director|manager" emp.lst  either word
$ grep "[[:digit:]]\{10\}" f       a ten-digit number
```

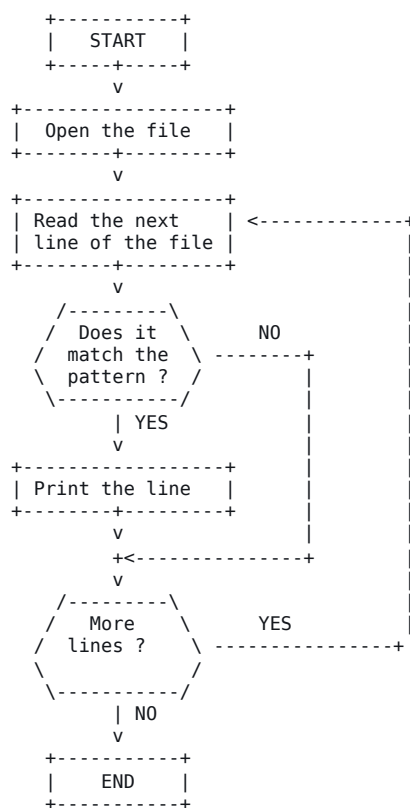
Marks tip: the examiner is looking for the **five headings** — Atoms, Quantifiers, Anchors, Alternation/Grouping, Character classes — with at least one example under each. Write the headings first, then fill them.

15. What is `grep` ? Explain the `grep` operation using a simple flow chart. Explain `grep` with its options. Differentiate between `grep` , `egrep` and `fgrep` .

LOCK Part B · Q7 or Q8 · 3–5 marks · Part B 3 of 3, Part A 3 of 3

grep stands for **Global Regular Expression Print**. It searches one or more files **line by line** for a pattern and prints every line that matches. It is the most used filter in UNIX.

Syntax: `grep [options] pattern filename(s)` — always **quote the pattern** so the shell does not expand its metacharacters.



Options

Option	Meaning	Example
<code>-i</code>	Ignore case when matching	<code>grep -i "unix" f</code>
<code>-v</code>	Invert — print lines that do not match	<code>grep -v "director" emp.lst</code>
<code>-c</code>	Print only the count of matching lines	<code>grep -c "unix" f</code>
<code>-n</code>	Prefix each line with its line number	<code>grep -n "main" prog.c</code>
<code>-l</code>	Print only the names of files containing a match	<code>grep -l "bash" *</code>
<code>-w</code>	Match the pattern as a whole word only	<code>grep -w "is" f</code>
<code>-x</code>	Match only if the whole line is the pattern	<code>grep -x "UNIX" f</code>
<code>-e</code>	Specify the pattern — allows several patterns	<code>grep -e "a" -e "b" f</code>

Option	Meaning	Example
-E	Use extended regular expressions (same as egrep)	grep -E "cat dog" f
-r	Search recursively through directories	grep -r "TODO" .

grep vs egrep vs fgrep

Command	Pattern type	Supports	Notes
grep	Basic regular expressions (BRE)	. * [] ^ \$ directly; + ? () only if escaped with \	The standard tool. Medium speed.
egrep	Extended regular expressions (ERE)	+ ? () { } work without backslashes	Same as grep -E. Most powerful, slowest.
fgrep	Fixed strings — no regular expressions at all	Nothing; every character is literal	Same as grep -F. Fastest , because there is no pattern matching to do. Use when searching for text containing * or . literally.

```
$ grep "director" emp.lst      lines containing "director"
$ grep -i "unix" f            case-insensitive
$ grep -c "unix" f            how many lines matched
$ grep -v "manager" emp.lst   everything EXCEPT managers
$ egrep "director|manager" emp.lst either word - needs egrep
$ fgrep "a*b" f               finds the literal text a*b
$ grep "^a.*z$" f             starts with a, ends with z
$ grep -l "bash" *            which files mention bash
```

16. Explain the tr command with its options / the various ways of using tr . Analyse the use of the uniq command with different options.

LOCK Part B · Q8 · 4-5 marks · tr 2 of 3 in the Q8(a) slot; uniq 1 of 3 + 2 Part A MCQs

tr — translate characters

tr copies standard input to standard output, **substituting or deleting characters** on the way. Syntax: tr [options] set1 set2

The exam point most students miss: tr is the only common filter that **takes no filename argument** — it reads **only from standard input**. You must use redirection < or a pipe.

Option	Meaning	Example
(none)	Translate each character of set1 into the character at the same position in set2	tr 'a-z' 'A-Z' < f
-d	Delete every character in set1	tr -d '0-9' < f
-s	Squeeze repeated consecutive characters into one	tr -s ' ' < f
-c	Complement — operate on every character not in set1	tr -cd '0-9' < f

```
$ tr 'a-z' 'A-Z' < file.txt      convert lowercase to uppercase
$ echo "hello" | tr 'a-z' 'A-Z'
HELLO

$ tr -d '0-9' < file.txt          remove all digits
$ tr -s ' ' < file.txt            squeeze multiple spaces into one
$ tr -s '\n' < file.txt           remove blank lines
$ tr ':' ' ' < /etc/passwd        replace every colon with a space
$ tr -cd '0-9' < file.txt         keep ONLY the digits (complement + delete)
$ cat file | tr '[:lower:]' '[:upper:]'
```

uniq — handle duplicate lines

uniq removes or reports **adjacent duplicate lines**. **Because it only ever compares a line with the one directly before it, the input must be sorted first** — this is why uniq is almost always used after sort .

Option	Meaning	Example
(none)	Remove duplicates, keeping one copy of each	<code>sort f uniq</code>
<code>-u</code>	Print only the lines that are not repeated at all	<code>sort f uniq -u</code>
<code>-d</code>	Print only the lines that are duplicated , once each	<code>sort f uniq -d</code>
<code>-c</code>	Prefix each line with the count of its occurrences	<code>sort f uniq -c</code>
<code>-i</code>	Ignore case when comparing	<code>sort f uniq -i</code>

```
$ cat names.txt          $ sort names.txt | uniq -c
amit                     2 amit
amit                     1 john
john                     3 ravi
ravi
ravi                     $ sort names.txt | uniq -d    -> amit, ravi
ravi                     $ sort names.txt | uniq -u    -> john
```

17. Explain the following filter commands with examples: `cat` , `head` , `tail` , `cut` , `paste` , `sort` , `wc` . Explain the commands used to compare two files.

HIGH Part B · Q7 · 3–5 marks · appears in all 3 papers, subset changes each time

A **filter** is a command that reads from standard input, transforms the data, and writes to standard output — which is what lets filters be chained with pipes.

Command	Purpose & options	Examples
cat	Concatenate / display / create files. <code>-n</code> number all lines · <code>-s</code> squeeze repeated blank lines · <code>-v</code> show non-printing characters · <code>-b</code> number non-blank lines	<code>cat f1 f2 > f3</code> join two files <code>cat > new.txt</code> create (Ctrl+D ends) <code>cat -n f</code> with line numbers
head	Display the beginning of a file. Default 10 lines. <code>-n k</code> or <code>-k</code> first k lines	<code>head -5 emp.lst</code> <code>head -n 3 f</code>
tail	Display the end of a file. Default 10 lines. <code>-n k</code> last k lines · <code>+k</code> from line k onward · <code>-f</code> follow a growing file	<code>tail -5 emp.lst</code> <code>tail +10 f</code> from line 10 <code>tail -f log.txt</code>
cut	Extract columns from each line. <code>-c</code> by character position · <code>-f</code> by field number · <code>-d</code> set the delimiter (default is Tab)	<code>cut -c 1-10 f</code> chars 1–10 <code>cut -c -3 f</code> chars 1–3 <code>cut -d: -f1,3 /etc/passwd</code>
paste	Join files side by side , line for line — the vertical counterpart of <code>cat</code> . <code>-d</code> set the delimiter	<code>paste f1 f2</code> <code>paste -d: f1 f2</code>
sort	Sort lines. Default delimiter is one or more spaces ; default order is ASCII. <code>-n</code> numeric · <code>-r</code> reverse · <code>-k n</code> sort on field n · <code>-t</code> field delimiter · <code>-u</code> unique · <code>-f</code> fold case	<code>sort names.txt</code> <code>sort -n marks.txt</code> <code>sort -t: -k3 -n /etc/passwd</code> <code>sort -r f</code> descending
wc	Word count — counts lines, words and characters. <code>-l</code> lines · <code>-w</code> words · <code>-c</code> characters	<code>wc file.txt</code> all three <code>wc -l file.txt</code> <code>who wc -l</code> users logged in

Why the counts matter (the `wc` question, Mar 2024)

Counting lines, words and characters is how you **measure** data without reading it: `wc -l` on a log tells you how many events occurred, `who | wc -l` tells you how many users are logged in, and `grep -c` combined with `wc` lets you quantify search results. Because `wc` is a filter it can end any pipeline, turning any output into a number.

Comparing two files

Command	What it does	Notes and options
<code>cmp</code>	Compares two files byte by byte and reports the first difference, giving its byte and line number.	Silent when the files are identical. Exit status 0 if identical , 1 if they differ. Works on binary files too.
<code>comm</code>	Compares two sorted files and prints three columns: lines only in file1, lines only in file2, lines in both.	<code>-1</code> <code>-2</code> <code>-3</code> suppress the corresponding column. <code>comm -12 f1 f2</code> shows only the common lines.
<code>diff</code>	Reports what changes would convert file1 into file2 , line by line, using <code>a</code> (add), <code>c</code> (change), <code>d</code> (delete).	<code>-c</code> context format · <code>-u</code> unified format · <code>-i</code> ignore case · <code>-b</code> ignore blank differences. Its output can be fed to <code>patch</code> .

```
$ cmp f1 f2
f1 f2 differ: byte 12, line 2

$ comm f1 f2          three columns: only-in-f1 | only-in-f2 | in-both
$ diff -u f1 f2       unified diff showing the edits needed
```

UNIT V — Shell Scripts

PART B · Q9 or Q10 · 8 marks | ⚠ no past-paper history

Why this unit has no "appeared in" line. In all three past papers the Unit V slot held **grep and regular expressions** — which your syllabus has moved into Unit IV. Shell scripting is new to this position, so the questions below are derived from the syllabus topic order and the paper's own 8-mark two-part structure. **Treat them as a strong, reasoned guess rather than a repeat.** They cover every construct the syllabus names, so the unit is fully covered whichever way it is split.

18. Explain the use of `test` and `[ ]` to evaluate expressions. Explain the `if` conditional with its forms.

UNIT V Part B · Q9(a) · 4–5 marks · first major construct named in the syllabus

The shell has no built-in comparison operators. **`test` is a command that evaluates an expression and returns an exit status — 0 (true) if the expression holds, non-zero (false) otherwise.** `[ ]` is simply another name for `test`.

```
test $a -gt $b      these two lines are
[ $a -gt $b ]       exactly equivalent
```

The rule that costs the most marks: `[` is a **command**, not punctuation — so it needs **spaces on both sides of everything**. `[ $a -gt $b ]` is correct; `[$a -gt $b]` is a syntax error.

Numeric comparison

Operator	True when	Example
<code>-eq</code>	equal to	<code>[\$a -eq \$b]</code>
<code>-ne</code>	not equal to	<code>[\$a -ne 0]</code>
<code>-gt</code>	greater than	<code>[\$a -gt \$b]</code>
<code>-ge</code>	greater than or equal to	<code>[\$a -ge 10]</code>
<code>-lt</code>	less than	<code>[\$a -lt \$b]</code>
<code>-le</code>	less than or equal to	<code>[\$a -le 100]</code>

String comparison

Operator	True when	Example
<code>=</code>	the two strings are identical	<code>["\$s1" = "\$s2"]</code>
<code>!=</code>	the strings differ	<code>["\$s1" != "abc"]</code>
<code>-z</code>	the string has zero length (empty)	<code>[-z "\$s"]</code>

Operator	True when	Example
<code>-n</code>	the string has non-zero length	<code>[-n "\$s"]</code>

Note: numeric comparison uses `-eq`; string comparison uses `=`. Mixing them up is the classic error. Always quote string variables so an empty value does not break the syntax.

File tests

Operator	True when the file...	Example
<code>-e</code>	exists	<code>[-e file]</code>
<code>-f</code>	exists and is a regular file	<code>[-f note.txt]</code>
<code>-d</code>	exists and is a directory	<code>[-d /home]</code>
<code>-r / -w / -x</code>	is readable / writable / executable	<code>[-r file]</code>
<code>-s</code>	exists and has size greater than zero	<code>[-s file]</code>

Logical operators

`-a` logical AND · `-o` logical OR · `!` logical NOT

`[ $a -gt 0 -a $a -lt 100 ]` — true when `a` is between 0 and 100

`[ ! -f file ]` — true when the file does **not** exist

The `if` conditional

`if` runs a command and branches on its **exit status** — 0 means the `then` branch runs. It is closed by `fi` (`if` reversed).

```
# Form 1 - simple if
if [ $a -gt $b ]
then
    echo "a is greater"
fi

# Form 2 - if...else
if [ $a -gt $b ]
then
    echo "a is greater"
else
    echo "b is greater or equal"
fi

# Form 3 - if...elif...else (multiway)
if [ $a -gt $b ]
then
    echo "$a is greater"
elif [ $a -eq $b ]
then
    echo "both are equal"
else
    echo "$b is greater"
fi
```

`if` can test any command, not just `test` :

```
if grep -q "unix" file.txt
then
    echo "Pattern found"
fi
# branches on grep's own exit status
```

19. Explain the `case` conditional with syntax and an example. Explain `expr` for computation and string handling.

UNIT V · Part B · Q10(a) · 4–5 marks · the classic menu-driven-script question

The `case` conditional

`case` compares one value against several **patterns** and runs the matching block. It is cleaner than a long `if-elif` chain when you are testing the same variable repeatedly, and unlike `if` it does **pattern** matching, so wildcards work.

```
case $variable in
  pattern1)
    commands
    ;;
  pattern2 | pattern3)      # several patterns, separated by |
    commands
    ;;
  *)                        # default - matches anything
    commands
    ;;
esac
```

- Each block **must end with** `;;` — this is what stops execution falling through to the next pattern.
- `*)` is the **default** case; put it last, as patterns are tried in order.
- The whole construct is closed by `esac` (`case` spelled backwards).
- Patterns may use shell wildcards: `[Yy]*)` matches yes, Yes, y, Y...

`expr` — computation and string handling

The shell treats everything as a string, so `expr` is the command that performs arithmetic. It evaluates an expression and prints the result.

Two rules that break scripts: every operand and operator must be **separated by spaces** (`expr 5+3` prints "5+3"; `expr 5 + 3` prints 8), and the multiplication sign must be **escaped** as `\*` because `*` is a shell wildcard.

```
$ expr 5 + 3      8      addition
$ expr 10 - 4     6      subtraction
$ expr 10 \* 2    20     multiplication - note the backslash
$ expr 10 / 3     3      integer division - remainder discarded
$ expr 10 % 3     1      modulus (remainder)

$ a=5 ; b=3
$ sum=`expr $a + $b`      store the result in a variable
$ echo $sum
8
$ sum=$(expr $a + $b)     the modern form does the same
```

String handling with `expr` :

Expression	Result	Meaning
<code>expr length "hello"</code>	5	Number of characters in the string
<code>expr substr "hello" 2 3</code>	ell	Substring: 3 characters starting at position 2
<code>expr index "hello" l</code>	3	Position of the first occurrence of the character

Worked example — menu-driven calculator (`case` + `expr` + `read`)

```
#!/bin/sh
# Menu-driven calculator using case and expr

echo "1. Add"
echo "2. Subtract"
echo "3. Multiply"
echo "4. Divide"
echo -n "Enter your choice: "
read ch
echo -n "Enter two numbers: "
read a b

case $ch in
  1) echo "Sum = `expr $a + $b`"
    ;;
  2) echo "Difference = `expr $a - $b`"
    ;;
  3) echo "Product = `expr $a \* $b`"
    ;;
  4) if [ $b -eq 0 ]
    then
      echo "Error: division by zero"
    else
      echo "Quotient = `expr $a / $b`"
    fi
    ;;
  *) echo "Invalid choice"
    ;;
esac
```

```
Sample run:
$ sh calc.sh
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice: 3
Enter two numbers: 6 7
Product = 42
```

20. Explain the use of command line arguments. Explain `exit` and the exit status of a command, and the logical operators `&&` and `||`. How is a script made interactive using `read`?

UNIT V Part B · Q9(b) · 3–4 marks · four syllabus lines, one answer

Making a script interactive — `read`

`read` takes a line from standard input and stores it in one or more variables, which is what turns a fixed script into an interactive one.

```
echo -n "Enter your name: "      # -n keeps the cursor on the same line
read name
echo "Hello, $name"

read a b c                      # one line split on whitespace into three variables
read                           # value goes into the default variable REPLY
```

Command line arguments (positional parameters)

Arguments typed after the script name are placed into **positional parameters**, so a script can be given data without prompting.

Parameter	Holds
<code>\$0</code>	The name of the script as it was invoked
<code>\$1 ... \$9</code>	The first through ninth arguments
<code>\$#</code>	The number of arguments passed (excludes <code>\$0</code>)
<code>\$*</code>	All arguments, as a single string
<code>@</code>	All arguments, as separate words — the safer of the two when quoted
<code>\$?</code>	Exit status of the last command executed
<code>\$\$</code>	The PID of the current shell

```
$ cat args.sh
#!/bin/sh
echo "Script name      : $0"
echo "First argument   : $1"
echo "Second argument  : $2"
echo "Number of args   : $# "
echo "All arguments    : $*"

$ sh args.sh unix shell programming
Script name      : args.sh
First argument   : unix
Second argument  : shell
Number of args   : 3
All arguments    : unix shell programming
```

`exit` and exit status

- **Every command returns an exit status** to the shell when it finishes — a number from 0 to 255.
- **0 means success. Any non-zero value means failure.** (This is the reverse of C's convention for truth, and is a favourite MCQ.)
- The status of the most recent command is held in `$?`. Read it **immediately** — the very next command overwrites it.
- `exit n` terminates a script and returns status `n` to whoever called it. `exit 0` conventionally signals success.

```
$ grep "unix" file.txt > /dev/null
$ echo $?
0                      # 0 -> the pattern was found

$ grep "xyz" file.txt > /dev/null
$ echo $?
1                      # non-zero -> not found
```

Logical operators `&&` and `||`

These chain commands based on the exit status of the one before, giving **conditional execution** without writing a full `if`.

Form	Behaviour
<code>cmd1 && cmd2</code>	Run <code>cmd2</code> only if <code>cmd1</code> succeeded (exit status 0)

Form	Behaviour
<code>cmd1 cmd2</code>	Run <code>cmd2</code> only if <code>cmd1</code> failed (non-zero status)
<pre>\$ grep "unix" file.txt && echo "Pattern found" \$ grep "xyz" file.txt echo "Pattern not found" \$ [-f data.txt] && echo "exists" echo "missing" # the combined idiom: if-then-else on a single line \$ mkdir project && cd project # only cd in if the directory was actually created</pre>	

21. Explain `while` looping and `for` looping with a list. Explain `set` and `shift` for manipulating positional parameters, and the use of `$0` to call a script by different names.

UNIT V Part B · Q10(b) · 3–4 marks · the last syllabus lines of the unit

`while` — loop while a condition holds

Repeats the body **as long as the condition is true** (exit status 0). Closed by `done`. Use it when the number of repetitions is not known in advance.

```
i=1
while [ $i -le 5 ]
do
    echo "Number is $i"
    i=`expr $i + 1`      # the counter MUST be incremented,
                        # or the loop never ends
done
```

Reading a file line by line is the other standard use:

```
while read line
do
    echo "Line: $line"
done < file.txt
```

`for` — loop over a list

Takes each item of a list in turn, assigns it to the variable, and runs the body once per item. Use it when the set of values is known.

```
for name in Amit John Ravi      # explicit list
do
    echo "Hello, $name"
done

for f in *.txt                  # wildcard expands to a file list
do
    echo "Processing $f"
    wc -l $f
done

for i in 1 2 3 4 5              # a numeric list
do
    echo "Square of $i is `expr $i \* $i`"
done

for user in `who | cut -d' ' -f1` # list from command substitution
do
    echo "Logged in: $user"
done

for arg in "$@"                 # loop over the script's arguments
do
    echo "Argument: $arg"
done
```

`set` — assigning positional parameters

`set` with arguments **assigns them to the positional parameters** `$1`, `$2`, `$3...`, overwriting whatever was there. This makes it a simple way to split a string into fields.

```

$ set 10 20 30
$ echo $1 $2 $3
10 20 30
$ echo $#
3

$ set `date`          # split date's output into fields
$ echo $1             Wed             (day)
$ echo $2             Sep             (month)
$ echo $6             2025            (year)

```

shift — moving the parameters left

`shift` discards `$1` and moves every other parameter one place to the left: `$2` becomes `$1`, `$3` becomes `$2`, and `$#` decreases by one. `shift n` shifts by `n` places.

Why it exists: only `$1` to `$9` are directly accessible. `shift` lets a script walk through **any** number of arguments by repeatedly examining `$1` and shifting.

```

$ set a b c d
$ echo $1 $2 $3 $4      -> a b c d
$ shift
$ echo $1 $2 $3         -> b c d      ("a" is gone, $# is now 3)
$ shift 2
$ echo $1               -> d

```

```

#!/bin/sh
# Print every argument, however many there are
echo "Total arguments: $#"
```

```

while [ $# -gt 0 ]
do
    echo "Argument: $1"
    shift
done

```

\$0 — calling a script by different names

`$0` holds **the name the script was actually invoked by**, not the name it was written as. By creating links to one script under several names and branching on `$0`, a **single script can behave differently depending on how it was called** — which is how several real UNIX utilities are implemented.

```

$ ln greet hello          # create links to the same script
$ ln greet bye

$ cat greet
#!/bin/sh
case $0 in
    *hello) echo "Hello! Welcome." ;;
    *bye)   echo "Goodbye! See you." ;;
    *)      echo "Usage: call me as hello or bye" ;;
esac

$ ./hello      -> Hello! Welcome.
$ ./bye        -> Goodbye! See you.

```

The advantage is a **single copy of the code** with shared logic, instead of two nearly identical scripts to maintain.

22. Write shell scripts for the following. (The lab component is shell script programs — expect at least one "write a script to..." part.)

UNIT V Part B · Q9(b) or Q10(b) · 3–5 marks each

(a) Find the largest of three numbers

```
#!/bin/sh
# Largest of three numbers - uses read, if-elif-else, test

echo -n "Enter three numbers: "
read a b c

if [ $a -gt $b -a $a -gt $c ]
then
    echo "$a is the largest"
elif [ $b -gt $c ]
then
    echo "$b is the largest"
else
    echo "$c is the largest"
fi

# Sample run:
# $ sh largest.sh
# Enter three numbers: 12 45 23
# 45 is the largest
```

(b) Check whether a number is even or odd

```
#!/bin/sh
# Even or odd - uses expr for modulus

echo -n "Enter a number: "
read n

r=`expr $n % 2`

if [ $r -eq 0 ]
then
    echo "$n is even"
else
    echo "$n is odd"
fi

# Sample run:
# Enter a number: 7
# 7 is odd
```

(c) Factorial of a number

```
#!/bin/sh
# Factorial - uses while and expr

echo -n "Enter a number: "
read n

f=1
i=1
while [ $i -le $n ]
do
    f=`expr $f \* $i`
    i=`expr $i + 1`
done

echo "Factorial of $n is $f"

# Sample run:
# Enter a number: 5
# Factorial of 5 is 120
```

(d) Reverse a number and find the sum of its digits

```
#!/bin/sh
# Reverse a number and sum its digits - while + expr

echo -n "Enter a number: "
read n

rev=0
sum=0
temp=$n

while [ $temp -gt 0 ]
do
    d=`expr $temp % 10`      # extract the last digit
    rev=`expr $rev \* 10 + $d` # build the reversed number
    sum=`expr $sum + $d`      # accumulate the digit sum
    temp=`expr $temp / 10`    # drop the last digit
done

echo "Reverse of $n is $rev"
echo "Sum of digits is $sum"

# Sample run:
# Enter a number: 1234
# Reverse of 1234 is 4321
# Sum of digits is 10
```

(e) Print the first n Fibonacci numbers

```
#!/bin/sh
# Fibonacci series - while + expr

echo -n "How many terms? "
read n

a=0
b=1
i=1

echo "Fibonacci series:"
while [ $i -le $n ]
do
    echo -n "$a "
    c=`expr $a + $b`
    a=$b
    b=$c
    i=`expr $i + 1`
done
echo

# Sample run:
# How many terms? 7
# Fibonacci series:
# 0 1 1 2 3 5 8
```

(f) Count the arguments passed and display each one

```
#!/bin/sh
# Walk through any number of arguments - $# and shift

if [ $# -eq 0 ]
then
    echo "Usage: $0 arg1 arg2 ..."
    exit 1
fi

echo "Number of arguments: $#"
```

echo "Script name: \$0"

```
n=1
while [ $# -gt 0 ]
do
    echo "Argument $n: $1"
    n=`expr $n + 1`
    shift
done
```

Sample run:

```
# $ sh args.sh unix shell programming
# Number of arguments: 3
# Script name: args.sh
# Argument 1: unix
# Argument 2: shell
# Argument 3: programming
```

(g) Check whether a file exists and report its permissions

```
#!/bin/sh
# File tests - -e -f -d -r -w -x

echo -n "Enter a filename: "
read fname

if [ ! -e $fname ]
then
    echo "$fname does not exist"
    exit 1
fi

if [ -d $fname ]
then
    echo "$fname is a directory"
elif [ -f $fname ]
then
    echo "$fname is a regular file"
fi

[ -r $fname ] && echo "Readable"
[ -w $fname ] && echo "Writable"
[ -x $fname ] && echo "Executable"
```

Sample run:

```
# Enter a filename: script.sh
# script.sh is a regular file
# Readable
# Writable
# Executable
```

(h) Count the number of files in a directory using a `for` loop

```
#!/bin/sh
# for looping with a list

count=0
for f in *
do
    if [ -f $f ]
    then
        echo "File: $f"
        count=`expr $count + 1`
    fi
done

echo "Total regular files: $count"
```

Marking traps — check every script you write against these:

- Start with `#!/bin/sh` • Use `expr` for arithmetic, **never** `$(( ))` • Use `[ ]`, **never** `[[ ]]`
- **Spaces are required** inside `[ ]` and around every `expr` operand • **No spaces** around `=` when assigning
- Escape multiplication as `\*` • Close with `fi`, `done`, `esac` • End every `case` branch with `;;`
- Increment the counter in a `while` loop, or it never terminates • Add a **sample run** as a comment — lab records need the output transcript

PART A — the ten 1-mark MCQs

10 × 1 = 10 marks · no choice · 2 per unit

Part A is **always two questions per unit, in unit order**. These are the facts the MCQs have actually tested, with the answer.

Slot	Unit	Question seen	Answer to remember
Q1	I	The UNIX shell is both ____ and ____ language	scripting, interpreter . Also: the shell is the interface between user and kernel.
Q2	I	Default permission for ordinary files / octal for <code>rw-r--r--</code> / components in a relative <code>chmod</code> expression	Default rw-rw-rw- (666) · 755 = <code>rw-r--r--</code> · 744 = <code>rw-r--r--</code> · relative mode has 3 components (who + operator + permission)
Q3	II	Command showing process attributes / option to create a tar archive	ps shows process attributes · tar -c creates, <code>-x</code> extracts, <code>-t</code> lists, <code>-f</code> names the file
Q4	II	Utility to test reachability of a host / command to manipulate the TCP/IP routing table / command to compare two files	ping tests reachability · route manipulates the routing table · diff / cmp / comm compare files
Q5	III	Symbol to connect different streams / to push a process to background / command that copies stdin to stdout and a file	<code> </code> connects streams (pipe) · & backgrounds a process · tee copies stdin to stdout <i>and</i> a file
Q6	III	Symbols that redirect output to a file or another program / keys to suspend a job	<code> </code> , <code>></code> and <code>>></code> — when "all of these" is an option it is usually correct · Ctrl+Z suspends a foreground job
Q7	IV	Option used with <code>cut</code> for fields / what <code>cut -c -3</code> cuts	-f = fields, -c = characters, -d = delimiter · <code>cut -c -3</code> gives columns 1-3
Q8	IV	Option with <code>uniq</code> for repeated / non-repeated lines · <code>sort</code> default delimiter · exit status of <code>cmp</code>	-d repeated, -u non-repeated, -c count · <code>sort</code> default delimiter is one or more spaces · <code>cmp</code> returns 0 if the files are identical
Q9	V	<i>No precedent — expect shell scripting</i> (old papers had <code>grep</code> here)	\$? = exit status of the last command, 0 = success · \$# = number of arguments · \$0 = script name · <i>grep fallback</i> : <code>-i</code> ignore case, <code>-c</code> count, <code>-v</code> invert
Q10	V	<i>No precedent — expect shell scripting</i> (old papers had <code>regex</code> here)	shift moves positional parameters left · esac closes <code>case</code> , fi closes <code>if</code> , done closes loops · -eq is numeric, <code>=</code> is string · <i>regex fallback</i> : <code>*</code> matches the immediately preceding character

each topic held a given question slot.

Stated plainly: Units I–IV rest on real repetition across three papers and are as reliable as three papers can make them. **Unit V has no precedent at all** — those questions and answers are derived from the syllabus topic list, so treat that section as thorough preparation of the required material rather than a prediction of the exact wording. Three papers is a small sample, and two of the three predate your syllabus revision.